Nombres

Lawrence Ortiz 2025-1116

Alber Rodriguez 2024-2259

Elizabeth Peralta 2025-1103

Luis Rainiel De Los Santos 2025-1052

Alison Moises Medina Ramírez 20252068

Asignatura

Programacion 1

Profesor

Jesus Maria Quezada

Proyecto

FleetFix

FleetFix

Sistema de gestión de flota vehicular

Documentación Técnica y Manual de Usuario

Nombre: **Luis Rainiel de los Santos**
Rol: Backend

Aporte:
Desarrolló la lógica interna del sistema, incluyendo el manejo de datos, conexión con la base de datos y funcionamiento de los tickets. Se encargó de que todas las operaciones del sistema funcionaran correctamente.

Nombre: **Elizabeth Peralta**
Rol: Base de Datos

Aporte:
Se encargó del diseño y creación de la base de datos, incluyendo el modelo entidad-relación y la estructura de las tablas. También definió las relaciones entre las entidades y aseguró que la información se almacenara de forma correcta.

Nombre: **Albert Rodríguez**
Rol: Frontend

Aporte:
Trabajó en el diseño de la interfaz gráfica del sistema, desarrollando las pantallas principales como login, dashboard y formularios. Se enfocó en que la aplicación fuera visualmente clara, fácil de usar y funcional para el usuario.

Nombre: **Lawrence Ortiz**
Rol: Documentación

Aporte:
Se encargó de elaborar toda la documentación del proyecto, organizando la información de manera clara y estructurada. Preparó la descripción del sistema, capturas de pantalla y explicó el funcionamiento general de la aplicación para su presentación.

Nombre: **Alison Moisés Medina Ramírez**
Rol: Prueba de Calidad

Aporte:
Se encargó de realizar pruebas al sistema para detectar errores y verificar que todas las funcionalidades trabajaran correctamente. Probó diferentes escenarios para asegurar la calidad y estabilidad de la aplicación.

Campo	Detalle
Version	1.0
Fecha	Abril 2026
Equipo	CodeStudents
Institucion	Instituto Tecnológico de Las Americas (ITLA)
Lenguaje	Java con Java Swing
Base de datos	SQLite (archivo fleetfix.db embebido)

1. Concepto y justificación de la solución

1.1 Concepto

FleetFix es una aplicación desarrollada para la gestión de incidencias en vehículos empresariales, permitiendo registrar tickets de fallas, clasificarlos según su nivel de prioridad y dar seguimiento a su estado.

1.2 Justificación

Se decidió implementar este sistema debido a la necesidad de llevar un control organizado de los problemas que presentan los vehículos de una empresa.
El uso de tickets permite identificar rápidamente fallas críticas, mejorar el mantenimiento preventivo y optimizar la gestión operativa.
Además, las tablas fueron diseñadas para evitar redundancia de datos y asegurar la integridad de la información, separando entidades como vehículos, tipos de vehículo, niveles y estados de ticket.

2. Introducción

FleetFix es un sistema de escritorio hecho en Java que sirve para gestionar los tickets de fallas de una flota de vehículos. La idea es que cualquier persona en la empresa pueda reportar cuando un vehículo tiene un problema, y que el administrador pueda darle seguimiento hasta que se resuelva.

El sistema tiene dos tipos de usuarios: el administrador, que puede hacer todo, y los usuarios normales, que pueden ver y crear tickets pero no cambiarles el estado.

2.1 Para qué sirve

Lo que hace el sistema es centralizar todos los reportes de fallas en un solo lugar. En vez de que cada persona anote las fallas en papel o en un chat, todo queda guardado con su nivel de prioridad, el vehiculo afectado y el estado actual.

2.2 Funciones principales

- Inicio de sesión con usuario y contraseña verificados en la base de datos
- Dashboard con números actualizados: tickets abiertos, en progreso, críticos y vehículos activos
- Crear tickets de fallas indicando el vehículo, prioridad, titulo y descripción
- Ver todos los tickets en una tabla con colores según el estado
- El administrador puede cambiar el estado de cualquier ticket
- Ver los vehículos registrados en la flota en formato de tarjetas

3. Como funciona el sistema

El sistema está dividido en dos partes principales: el backend, que maneja la base de datos y la lógica del programa, y el frontend, que es todo lo que el usuario ve en pantalla. Estas dos partes están separadas para que sea mas fácil modificar cualquiera de las dos sin romper la otra.

3.1 Flujo general

Básicamente el sistema funciona así:

- El usuario abre el programa y aparece la pantalla de login
- Escribe su usuario y contraseña, el sistema los verifica en la base de datos SQLite
- Si están correctos, entra al Dashboard con todas las métricas actuales
- Desde el menú lateral puede navegar entre las secciones del sistema
- Puede crear tickets nuevos o consultar los que ya existen
- El administrador además puede cambiarle el estado a cualquier ticket

3.2 Permisos por tipo de usuario

Funcion	Usuario Normal	Administrador
Ver el dashboard	Si	Si
Ver la lista de tickets	Si	Si
Crear un ticket nuevo	Si	Si
Cambiar el estado de un ticket	No	Si
Ver el inventario de vehículos	Si	Si
Cerrar sesion	Si	Si

4. Tecnologías usadas

El sistema se hizo con Java puro, sin librerías externas de diseño. Esto hace que sea fácil de instalar porque no depende de nada complicado. La base de datos es SQLite, que es un archivo local que se crea automáticamente dentro de la carpeta del programa, así que no hay que instalar ningún servidor de base de datos por separado.

Tecnologia	Para que se usa	Version
Java	Lenguaje principal del sistema	JDK 11 o superior
Java Swing	Para construir la interfaz grafica	Incluida en JDK
JDBC	Para conectar Java con la base de datos	Incluida en JDK
SQLite	Base de datos local, se guarda como archivo .db	3.x
sqlite-jdbc	El driver que conecta Java con SQLite	3.36.0.3
Apache NetBeans	IDE donde se desarrollo el proyecto	17 o superior
Graphics2D	Para el diseño personalizado de la interfaz	Incluida en JDK

5. Requisitos para ejecutar el programa

5.1 Software necesario

Software	Version	Donde descargarlo
Java JDK	11 o superior	oracle.com/java/technologies/downloads/
Apache NetBeans	17 o superior	netbeans.apache.org/front/main/download/
sqlite-jdbc	3.36.0.3	github.com/xerial/sqlite-jdbc/releases

5.2 Hardware minimo

- Procesador: 1 GHz o superior
- Memoria RAM: 1 GB mínimo (2 GB recomendado)
- Espacio en disco: 100 MB libres (SQLite no requiere servidor)
- Sistema operativo: Windows 10 o superior

Nota: Como la base de datos es SQLite, el archivo fleetfix.db se crea solo la primera vez que se ejecuta el programa. No hay que instalar ni configurar ningún servidor de base de datos.

6. Estructura del proyecto

El proyecto tiene dos paquetes principales. Uno es el backend, que maneja todo lo de la base de datos, y el otro es el frontend, que es la parte visual. Esta separación sigue el patrón DAO (Data Access Object), que es básicamente una forma ordenada de organizar el código para que cada clase se encargue de una sola cosa.

6.1 Backend (acceso a datos)

Archivo	Que hace
Conexion.java	Se conecta a SQLite y crea el archivo fleetfix.db si no existe. Tambien crea las tablas e inserta los datos de prueba al arrancar por primera vez.
UsuarioDAO.java	Verifica si el usuario y contrasena que escribio el usuario son correctos.
TicketDAO.java	Crea tickets nuevos, los consulta todos y permite cambiar su estado. Si el nivel es Critico, lo marca automaticamente.
VehiculoDAO.java	Lista los vehiculos que estan en la base de datos y cuenta cuantos estan operativos para el dashboard.

6.2 Frontend (interfaz grafica)

Archivo	Que hace
Tema.java	Tiene todos los colores y fuentes que usa la interfaz para que todo el diseno sea consistente.
LoginFrame.java	La primera pantalla que aparece al abrir el programa.
DashboardFrame.java	La ventana principal con el menu lateral y el area de contenido al centro.
NavButton.java	El componente visual de los botones del menu lateral.
PaginaDashboard.java	Muestra las metricas en tiempo real y la tabla de los ultimos tickets.
PaginaTickets.java	Lista completa de tickets. El admin ve el boton para cambiar estado.
PaginaNuevoTicket.java	El formulario para registrar una falla nueva.
PaginaVehiculos.java	Muestra los vehiculos en tarjetas con su informacion y cantidad de tickets.

7. Base de datos

El sistema usa SQLite como base de datos. A diferencia de MySQL o PostgreSQL, SQLite no necesita un servidor corriendo por separado, sino que todo se guarda en un archivo llamado fleetfix.db que queda dentro de la carpeta del proyecto. Esto hace que el sistema sea muy facil de mover de una computadora a otra.

La base de datos tiene 6 tablas que están conectadas entre si. Básicamente los tickets son el centro de todo, y las otras tablas les dan información como el tipo de vehículo, el nivel de prioridad, el estado actual y quien lo creo.

7.1 Diagrama Entidad-Relación (ER)

El siguiente diagrama muestra como están relacionadas las 6 tablas del sistema:

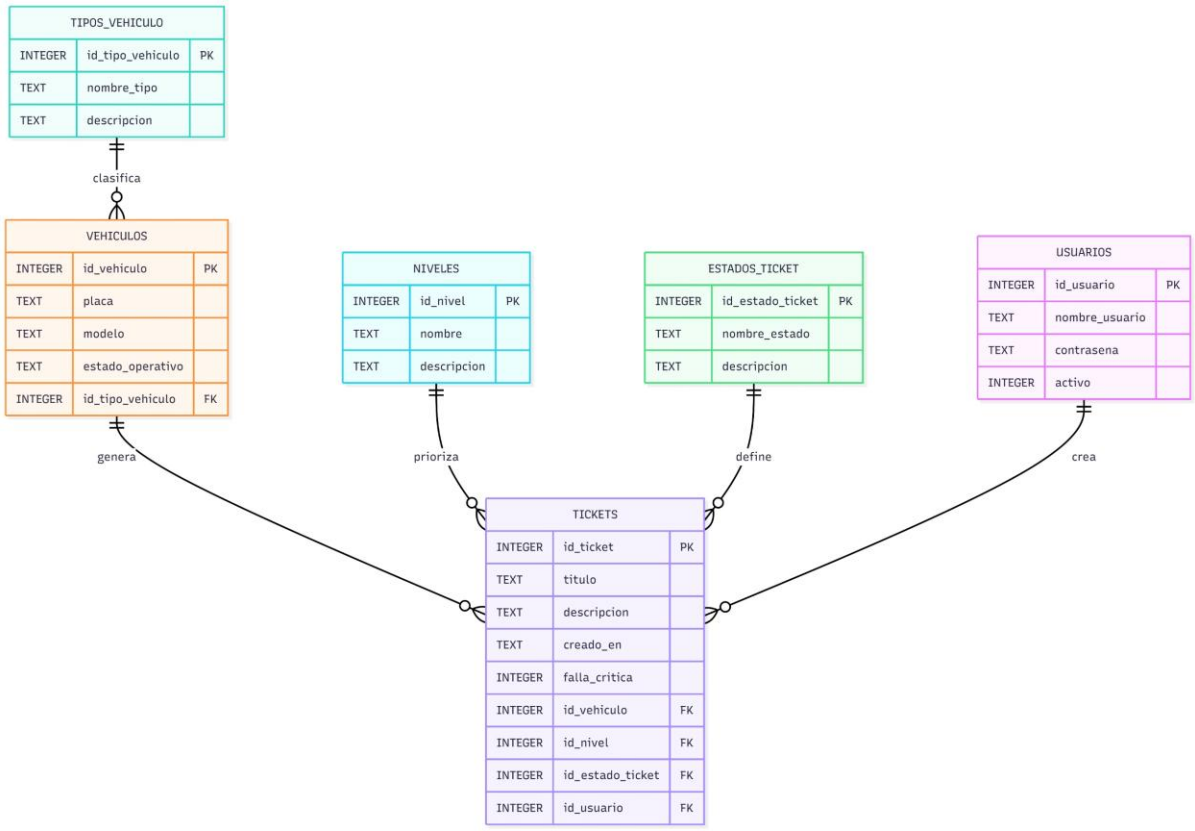


Diagrama ER del sistema FleetFix — muestra las 6 tablas y sus relaciones

7.2 Relaciones entre las tablas

- TIPOS_VEHICULO clasifica a VEHICULOS — un tipo puede tener muchos vehiculos
- VEHICULOS genera TICKETS — un vehiculo puede tener varios tickets abiertos
- NIVELES prioriza TICKETS — cada ticket tiene un nivel: Bajo, Medio, Alto o Critico
- ESTADOS_TICKET define el estado de cada TICKET — Abierto, En progreso, Resuelto o Cerrado
- USUARIOS crea TICKETS — cada ticket queda asociado al usuario que lo registro

7.3 Descripción de las tablas

TIPOS_VEHICULO

Columna	Tipo	Descripcion
id_tipo_vehiculo	INTEGER (PK)	Identificador unico del tipo de vehiculo
nombre_tipo	TEXT	Nombre: SUV, Sedan, Pickup, Coupe, Deportivo, etc.
descripcion	TEXT	Descripcion adicional del tipo (opcional)

VEHICULOS

Columna	Tipo	Descripcion
id_vehiculo	INTEGER (PK)	Identificador unico del vehiculo
placa	TEXT	Numero de placa del vehiculo
modelo	TEXT	Marca y modelo
estado_operativo	TEXT	Operativo o En revision
id_tipo_vehiculo	INTEGER (FK)	Referencia a TIPOS_VEHICULO

NIVELES

Columna	Tipo	Descripcion
id_nivel	INTEGER (PK)	ID del nivel (1=Bajo, 2=Medio, 3=Alto, 4=Critico)
nombre	TEXT	Nombre del nivel de prioridad
descripcion	TEXT	Que significa ese nivel de urgencia

ESTADOS_TICKET

Columna	Tipo	Descripcion
id_estado_ticket	INTEGER (PK)	Identificador unico del estado
nombre_estado	TEXT	Abierto, En progreso, Resuelto o Cerrado
descripcion	TEXT	Que significa cada estado

USUARIOS

Columna	Tipo	Descripcion
id_usuario	INTEGER (PK)	Identificador unico del usuario
nombre_usuario	TEXT	Nombre de login (unico en el sistema)
contraseña	TEXT	Contraseña del usuario
activo	INTEGER	1 = activo, 0 = desactivado

TICKETS

Columna	Tipo	Descripcion
id_ticket	INTEGER (PK)	Identificador Unico del ticket
titulo	TEXT	Titulo breve de la falla
descripcion	TEXT	Descripción detallada de lo que pasa
creado_en	TEXT	Fecha y hora de creación
falla_critica	INTEGER	1 si el nivel es Critico, 0 si no
id_vehiculo	INTEGER (FK)	Vehículo que tiene la falla
id_nivel	INTEGER (FK)	Nivel de prioridad del ticket
id_estado_ticket	INTEGER (FK)	Estado actual del ticket
id_usuario	INTEGER (FK)	Usuario que creo el ticket

8. Backend — Clases DAO

El patrón DAO (Data Access Object) es básicamente una forma de separar el codigo que habla con la base de datos del resto del programa. Cada clase DAO se encarga de una tabla especifica, lo que hace que sea mas fácil organizar y modificar el código.

8.1 Conexion.java

Esta clase maneja la conexión con SQLite. Cuando el programa arranca por primera vez, crea el archivo fleetfix.db y genera todas las tablas automáticamente. También inserta los datos iniciales de prueba para que el sistema ya tenga algo con que trabajar desde el primer momento.

```
Connection con = Conexion.getConexion();
```

8.2 UsuarioDAO.java

Tiene el metodo validarLogin() que recibe el usuario y contraseña, los busca en la tabla USUARIOS y devuelve true si son correctos y el usuario esta activo.

```
boolean ok = new UsuarioDAO().validarLogin(usuario, contraseña);
```

8.3 TicketDAO.java

Esta es la clase más importante. Maneja todo lo relacionado con los tickets:

Metodo	Que hace
crearTicket(...)	Inserta un ticket nuevo. Si el nivel es Critico, pone falla critica = 1 automáticamente.
obtenerTodos()	Devuelve todos los tickets listos para mostrar en la tabla de la interfaz.
cambiarEstado(id, estado)	Actualiza el estado de un ticket. Solo el admin puede usar esto.
contarPorEstado(estado)	Cuenta cuantos tickets hay en un estado específico para las métricas.
contarCriticos()	Cuenta los tickets críticos para la tarjeta del dashboard.

8.4 VehiculoDAO.java

Metodo	Que hace
obtenerTodos()	Devuelve todos los vehículos para mostrarlos en la página de vehículos.
contarActivos()	Cuenta los vehículos operativos para la tarjeta del dashboard.

9. Frontend — Interfaz grafica

La interfaz fue hecha completamente con Java Swing. El diseño oscuro con detalles en amarillo se construyó usando Graphics2D sin depender de librerías externas, lo que hace que el programa funcione en cualquier computadora que tenga Java instalado sin necesidad de instalar nada más.

9.1 Pantallas del sistema

Pantalla	Descripcion
LoginFrame	Primera pantalla al abrir el programa. Si las credenciales son incorrectas, la ventana se sacude como animación de error.
DashboardFrame	Ventana principal con el menú lateral y el área de contenido al centro.
PaginaDashboard	Muestra 4 tarjetas de métricas y la tabla con los últimos 5 tickets.
PaginaTickets	Lista completa de tickets con colores por estado. El admin ve el botón de cambiar estado.
PaginaNuevoTicket	Formulario para registrar una falla. Se escoge el vehículo, nivel y se describe el problema.
PaginaVehiculos	Muestra los vehículos en tarjetas con cuantos tickets tienen asociados.

9.2 Estados de un ticket

Estado	Que significa
Abierto	El ticket fue creado y está esperando que alguien lo atienda.
En progreso	Ya hay alguien trabajando en esa falla.
Resuelto	La falla fue arreglada y el vehículo ya esta operativo.
Cerrado	El ticket se cerró y ya no necesita ninguna acción.

10. Manual de usuario

10.1 Iniciar sesión

Al abrir el programa aparece la pantalla de login. Ahí el usuario escribe sus credenciales para entrar al sistema.

1. Abrir el programa — aparece la pantalla de login automáticamente
2. Escribir el nombre de usuario
3. Escribir la contraseña Presionar el botón Iniciar sesión o la tecla Enter
4. Si las credenciales están mal, aparece un mensaje de error y la ventana se sacude — intentar de nuevo
5. Si están correctas, el sistema abre el dashboard con las métricas del momento

10.2 Ver el dashboard

Al entrar aparece el dashboard con 4 tarjetas: tickets abiertos, en progreso, críticos y vehículos activos. Abajo hay una tabla con los últimos 5 tickets registrados. Todo se actualiza cada vez que el usuario vuelve a esta pantalla.

10.3 Registrar un nuevo ticket

6. Hacer clic en 'Nuevo ticket' en el menú lateral
7. Seleccionar el tipo de vehículo del menú desplegable
8. Seleccionar el nivel de prioridad: Bajo, Medio, Alto o Critico
9. Escribir un título breve que describa la falla
10. Escribir una descripción más detallada de lo que pasa
11. Hacer click en 'Crear ticket'

Nota: Si el nivel seleccionado es Critico, el ticket se marca automáticamente como falla crítica y aparece resaltado en las métricas del dashboard.

10.4 Gestionar tickets (solo administrador)

12. Hacer clic en 'Tickets' en el menú lateral
13. Seleccionar el ticket que se quiere actualizar haciendo clic sobre el
14. Hacer clic en el botón 'Cambiar estado' (esquina superior derecha)
15. Seleccionar el nuevo estado: Abierto, En progreso, Resuelto o Cerrado
16. Hacer clic en OK para confirmar

Nota: El botón 'Cambiar estado' solo aparece para el administrador. Los usuarios normales solo pueden ver los tickets.

10.5 Ver vehículos

Hacer clic en 'Vehículos' en el menú lateral. Se muestra una cuadricula con tarjetas de cada tipo de vehículo que tiene tickets registrados, junto con la cantidad de tickets de cada uno.

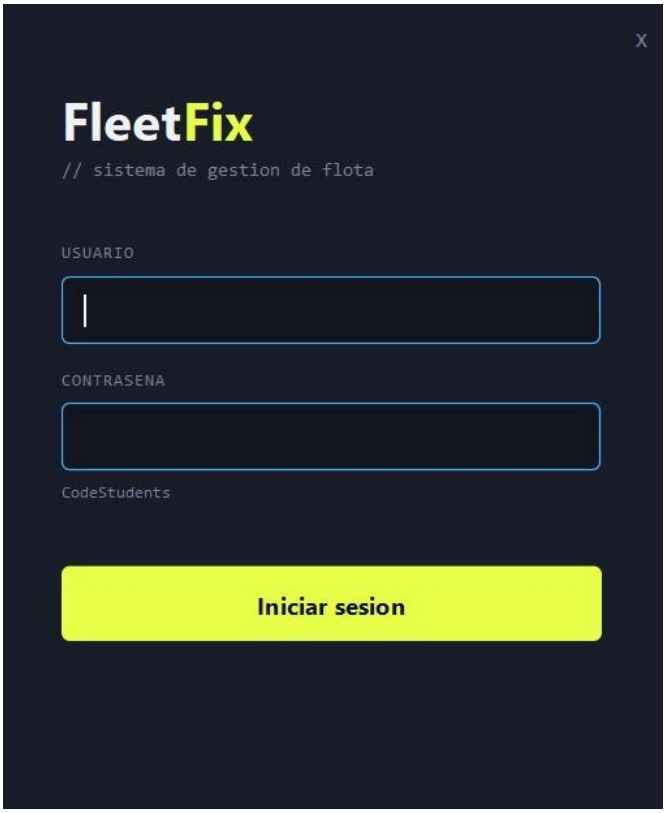
10.6 Cerrar sesión

Hacer clic en 'Cerrar sesión' en la parte inferior del menú lateral. El sistema regresa a la pantalla de login.

11. Capturas del programa

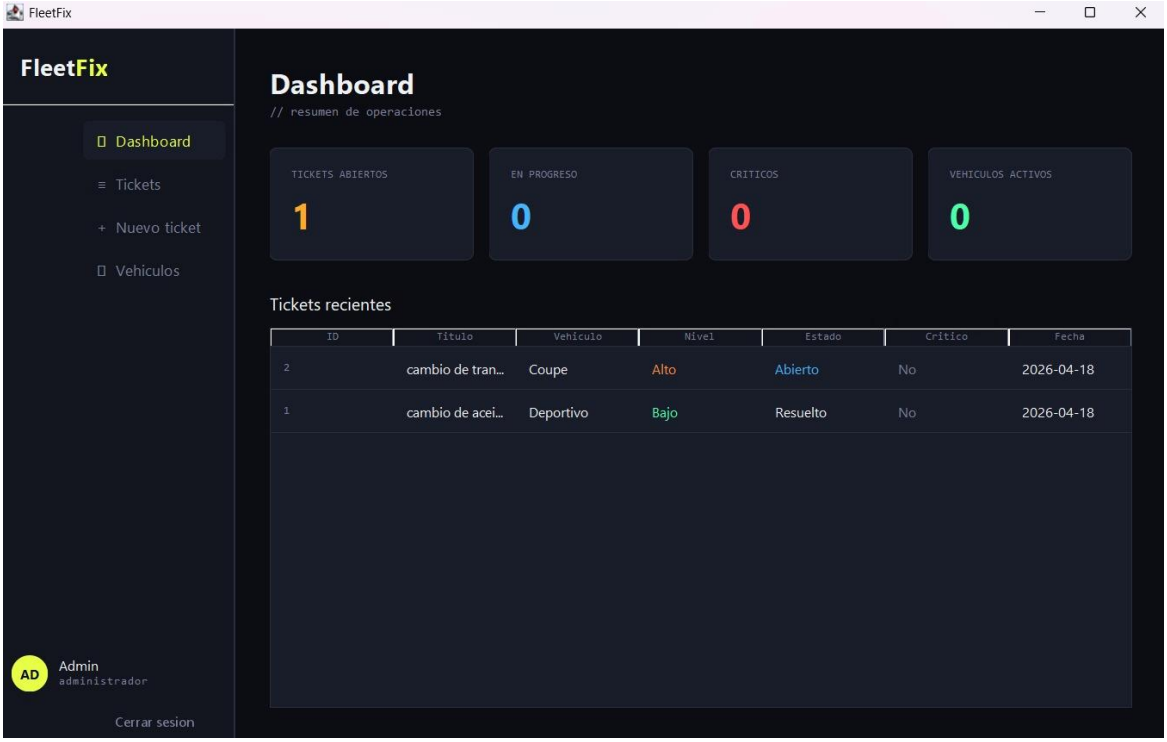
Estas son las pantallas principales del sistema tal como se ven al ejecutarlo.

Figura 1 — Pantalla de inicio de sesión



Pantalla de login. Diseño oscuro con el logo FleetFix, dos campos de entrada y el botón amarillo para iniciar sesión.

Figura 2 — Dashboard principal



Dashboard con las 4 tarjetas de métricas en tiempo real y la tabla de tickets recientes.

Figura 3 — Lista de tickets

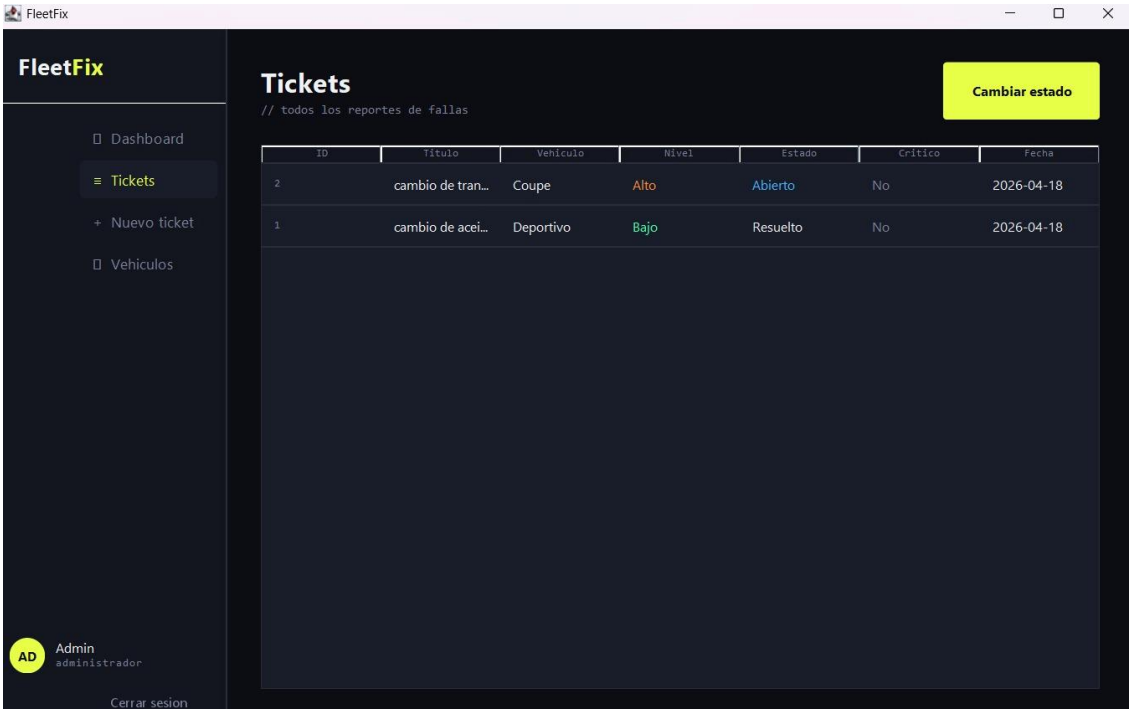
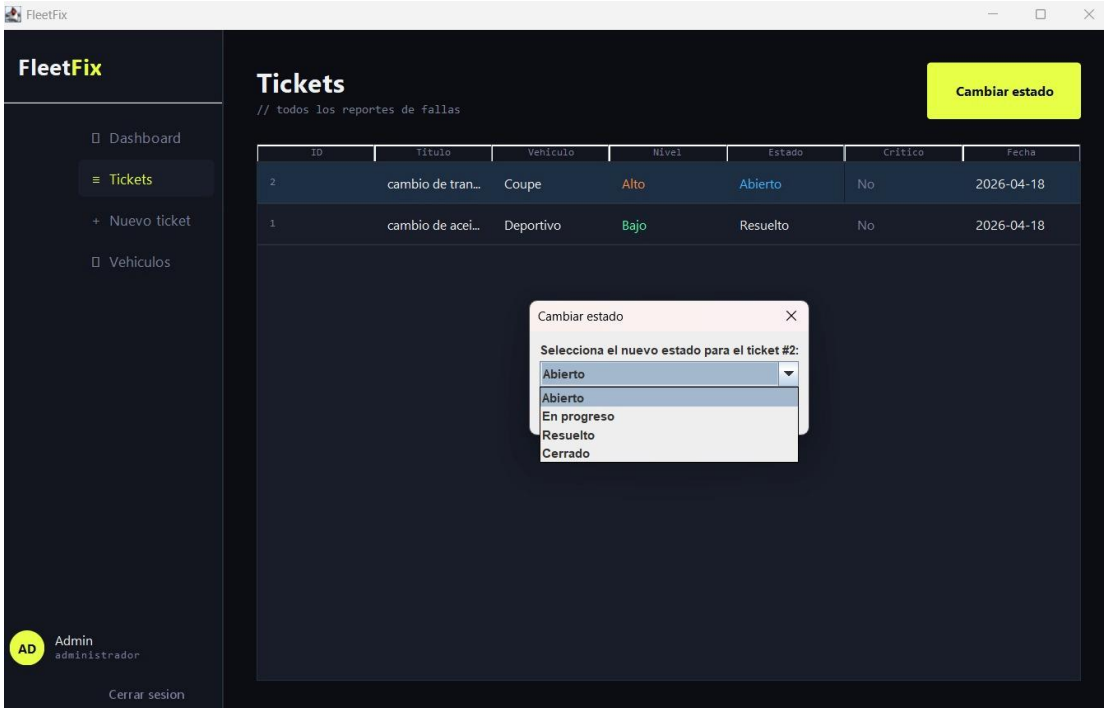


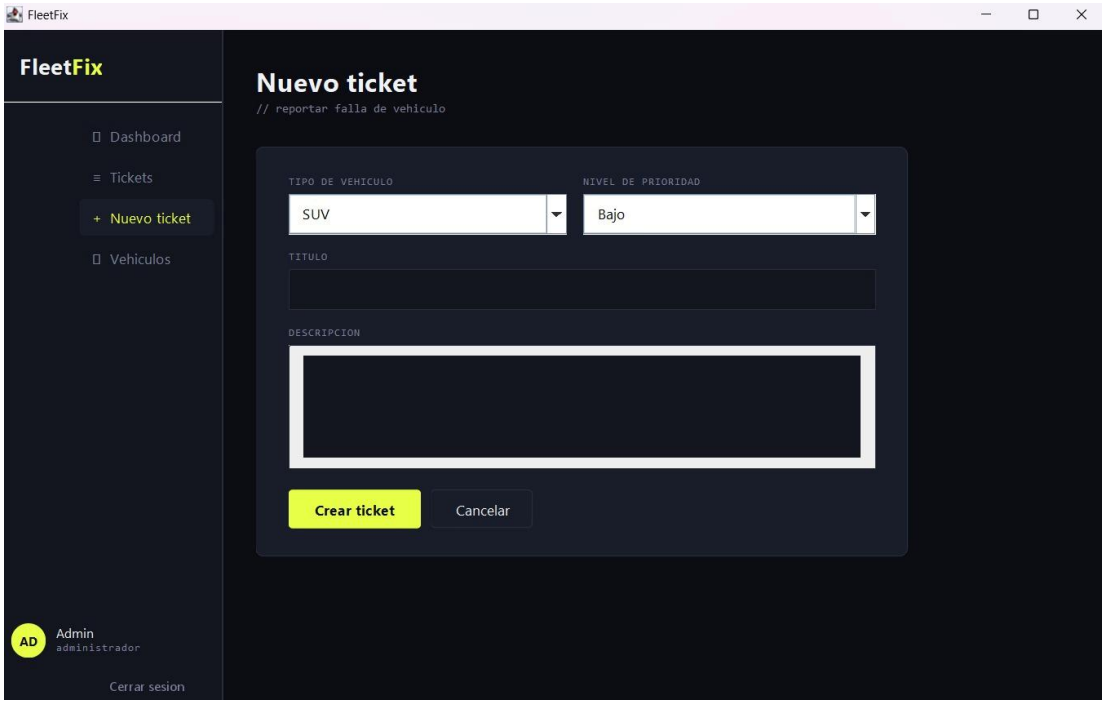
Tabla completa de tickets. Los colores en las columnas Nivel y Estado facilitan la lectura rápida.

Figura 4 — Cambio de estado de un ticket



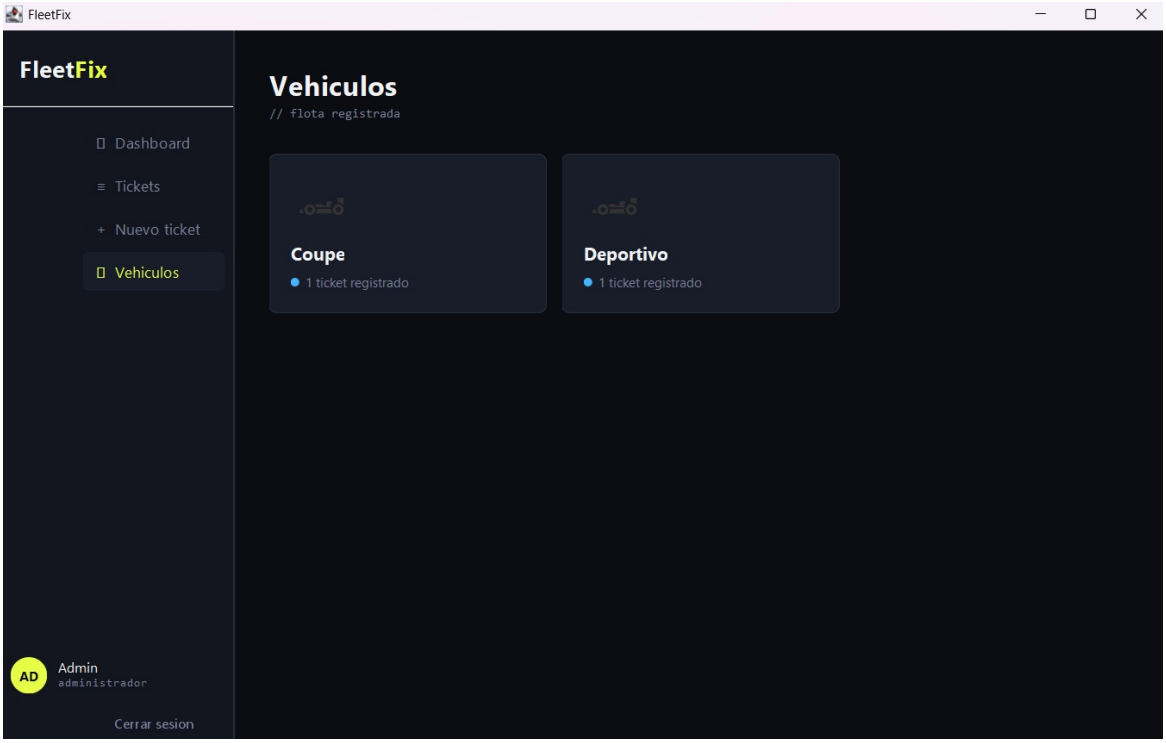
Dialogo para cambiar el estado de un ticket. Solo visible y accesible para el administrador.

Figura 5 — Formulario de nuevo ticket



Formulario para registrar una falla. El usuario escoge el tipo de vehículo, el nivel de prioridad, y describe el problema.

Figura 6 — Modulo de vehiculos



Sección de vehículos en tarjetas. Se muestran solo los tipos que tienen tickets registrados, con la cantidad de cada uno.

12. Credenciales y datos de prueba

La primera vez que se ejecuta el programa, el sistema crea la base de datos e inserta automáticamente datos de prueba. Esto permite usar el sistema de inmediato sin tener que configurar nada a mano.

12.1 Usuarios del Sistema

Usuario	Contraseña	Rol	Que puede hacer
admin	admin123	Administrador	Acceso completo, incluyendo cambiar el estado de los tickets
raini	fleetfix	Usuario normal	Ver y crear tickets, consultar vehiculos

12.2 Niveles de prioridad

ID	Nombre	Descripción
1	Bajo	Falla menor que no afecta la operación. Atender cuando sea posible.
2	Medio	Requiere atención pronto pero el vehiculo sigue operando.
3	Alto	Afecta el rendimiento. El vehículo puede seguir con limitaciones.
4	Critico	Falla grave que impide usar el vehículo. Se marca automáticamente como critico.

12.3 Estados disponible

Estado	Descripción
Abierto	Estado inicial. El ticket fue creado y está pendiente de atención.
En progreso	El equipo técnico esta trabajando en la falla.
Resuelto	La falla fue corregida y el vehículo ya esta operativo.
Cerrado	El ticket fue cerrado y archivado. No requiere mas accion.

Link Del Repositorio: https://github.com/Rg4ar/FleetFix